

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate

7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I, Judy Allen J (192520055) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Judy Allen J

1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply

Tables

EnrollmentID	StudentID	Subject	Credits	Semester
101	1	Database Systems	3	Fall 2025
102	2	Calculus	4	Fall 2025
103	3	Programming	3	Fall 2025
104	1	Data Structures	3	Fall 2025
105	4	Mechanics	4	Fall 2025
106	5	Statistics	3	Fall 2025

StudentId	StudentName	Course	Grade
1	Alice Johnson	Computer Science	A
2	Bob Smith	Mathematics	B
3	Charlie Brown	Computer Science	C
4	Diana Williams	Physics	A
5	Evan Davis	Mathematics	B
6	Fiona Miller	Computer Science	A

Query using Join:

```
mysql> SELECT
```

```
-> s.StudentName,
```

```
-> s.Course,
```

```
-> s.Grade,
```

```
-> en.Subject,
```

```
-> en.Credits,
```

```
-> en.Semester
```

```
-> FROM Students s
```

```
-> JOIN Enrollments en ON s.StudentID = en.StudentID
```

```
-> ORDER BY s.StudentName;
```

```
mysql> SELECT
-> s.StudentName,
-> s.Course,
-> s.Grade,
-> en.Subject,
-> en.Credits,
-> en.Semester
-> FROM Students s
-> JOIN Enrollments en ON s.StudentID = en.StudentID
-> ORDER BY s.StudentName;
```

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

6 rows in set (0.00 sec)

Output:

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

Logic:

Take student details from the Students table, match them with their enrollment records using StudentID, display selected columns from both tables, and arrange everything alphabetically by student name.

Query using Subquery:

```
mysql> SELECT
-> StudentID,
-> StudentName,
-> Course,
-> Grade
-> FROM Students
-> WHERE StudentID IN (
-> SELECT StudentID
-> FROM Enrollments
```

-> GROUP BY StudentID

-> HAVING COUNT(*) > 1

->);

```
mysql> SELECT
-> StudentID,
-> StudentName,
-> Course,
-> Grade
-> FROM Students
-> WHERE StudentID IN (
-> SELECT StudentID
-> FROM Enrollments
-> GROUP BY StudentID
-> HAVING COUNT(*) > 1
-> );
```

StudentID	StudentName	Course	Grade
1	Alice Johnson	Computer Science	A

1 row in set (0.04 sec)

Output:

StudentID	StudentName	Course	Grade
1	Alice Johnson	Computer Science	A

Logic:

First find which students are enrolled in more than one subject by counting their enrollments, then show the complete details of only those students from the Students table.

2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Bloom's Level: BL4 – Analyse

Notation:

π StudentName (

σ Course='Computer Science' (Students)

$\bowtie$

σ Subject='Database Systems' (Enrollments)

)

3. Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create

Table:

EnrollmentID	StudentID	Subject	Credits	Semester
101	1	Database Systems	3	Fall 2025
102	2	Calculus	4	Fall 2025
103	3	Programming	3	Fall 2025
104	1	Data Structures	3	Fall 2025
105	4	Mechanics	4	Fall 2025
106	5	Statistics	3	Fall 2025

StudentId	StudentName	Course	Grade
1	Alice Johnson	Computer Science	A
2	Bob Smith	Mathematics	B
3	Charlie Brown	Computer Science	C
4	Diana Williams	Physics	A
5	Evan Davis	Mathematics	B
6	Fiona Miller	Computer Science	A

Query for Data Retrieval:

mysql> SELECT

-> s.StudentName,

-> s.Course,

-> s.Grade,

-> en.Subject,

-> en.Credits,

-> en.Semester

-> FROM Students s

-> JOIN Enrollments en ON s.StudentID = en.StudentID

-> ORDER BY s.StudentName;

```
mysql> SELECT
-> s.StudentName,
-> s.Course,
-> s.Grade,
-> en.Subject,
-> en.Credits,
-> en.Semester
-> FROM Students s
-> JOIN Enrollments en ON s.StudentID = en.StudentID
-> ORDER BY s.StudentName;
```

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

6 rows in set (0.00 sec)

Output:

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

Query for Data Update:

```
mysql> UPDATE Students
```

```
-> SET Course = 'Data Science'
```

```
-> WHERE StudentID = 1;
```

```
mysql> SELECT * FROM Students ORDER BY StudentID;
```

```
mysql> UPDATE Students
-> SET Course = 'Data Science'
-> WHERE StudentID = 1;
Query OK, 1 row affected (0.03 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM Students WHERE StudentID = 1;
+----+-----+-----+-----+
| StudentID | StudentName | Course | Grade |
+----+-----+-----+-----+
| 1 | Alice Johnson | Data Science | A |
+----+-----+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT * FROM Students ORDER BY StudentID;
+----+-----+-----+-----+
| StudentID | StudentName | Course | Grade |
+----+-----+-----+-----+
| 1 | Alice Johnson | Data Science | A |
| 2 | Bob Smith | Mathematics | B |
| 3 | Charlie Brown | Computer Science | C |
| 4 | Diana Williams | Physics | A |
| 5 | Evan Davis | Mathematics | B |
| 6 | Fiona Miller | Computer Science | A |
+----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Output:

StudentId	StudentName	Course	Grade
1	Alice Johnson	Data Science	A
2	Bob Smith	Mathematics	B
3	Charlie Brown	Computer Science	C
4	Diana Williams	Physics	A
5	Evan Davis	Mathematics	B

6	Fiona Miller	Computer Science	A
---	--------------	------------------	---

4. Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze

Sample Query:

```
mysql> SELECT StudentName, Course, Grade
      -> FROM Students
      -> WHERE StudentID IN (
      ->   SELECT StudentID
      ->   FROM Enrollments
      ->   WHERE Credits > 3
      -> );
```

```
mysql> SELECT StudentName, Course, Grade
      -> FROM Students
      -> WHERE StudentID IN (
      ->   SELECT StudentID
      ->   FROM Enrollments
      ->   WHERE Credits > 3
      -> );
```

StudentName	Course	Grade
Bob Smith	Mathematics	B
Diana Williams	Physics	A

2 rows in set (0.00 sec)

Execution Steps:

Step-1

Inner subquery runs first: Finds StudentID where Credits > 3 → Returns (2, 4)

Step-2

Outer query runs second: Finds students with StudentID IN (2, 4) → Returns Bob Smith and Diana Williams

Step-3

Final result shows: Students enrolled in courses with more than 3 credits

Output:

StudentName	Course	Grade
Bob Smith	Mathematics	B
Diana Williams	Physics	A

5. Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate

Problem:

Find students who are enrolled in more than one subject and display their details along with enrollment information.

Query 1: JOIN Approach

```
mysql> SELECT
```

```
-> s.StudentName,
```

```
-> s.Course,
```

```
-> s.Grade,
```

```
-> en.Subject,
```

```
-> en.Credits,
```

```
-> en.Semester
```

```
-> FROM Students s
```

```
-> JOIN Enrollments en ON s.StudentID = en.StudentID
```

```
-> ORDER BY s.StudentName;
```

```
mysql> SELECT
-> s.StudentName,
-> s.Course,
-> s.Grade,
-> en.Subject,
-> en.Credits,
-> en.Semester
-> FROM Students s
-> JOIN Enrollments en ON s.StudentID = en.StudentID
-> ORDER BY s.StudentName;
```

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

```
6 rows in set (0.00 sec)
```

Output:

This approach shows ALL students with their enrollments

StudentName	Course	Grade	Subject	Credits	Semester
Alice Johnson	Computer Science	A	Database Systems	3	Fall 2025
Alice Johnson	Computer Science	A	Data Structures	3	Fall 2025
Bob Smith	Mathematics	B	Calculus	4	Fall 2025
Charlie Brown	Computer Science	C	Programming	3	Fall 2025
Diana Williams	Physics	A	Mechanics	4	Fall 2025
Evan Davis	Mathematics	B	Statistics	3	Fall 2025

Query 2: Subquery Approach

```
mysql> SELECT
```

```
-> StudentID,
```

```
-> StudentName,
```

```
-> Course,
```

```
-> Grade
```

```
-> FROM Students
```

```
-> WHERE StudentID IN (
```

```
-> SELECT StudentID
```

```
-> FROM Enrollments
```

```
-> GROUP BY StudentID
```

```
-> HAVING COUNT(*) > 1
```

```
-> );
```

```
mysql> SELECT
-> StudentID,
-> StudentName,
-> Course,
-> Grade
-> FROM Students
-> WHERE StudentID IN (
-> SELECT StudentID
-> FROM Enrollments
-> GROUP BY StudentID
-> HAVING COUNT(*) > 1
-> );
+-----+-----+-----+-----+
| StudentID | StudentName | Course | Grade |
+-----+-----+-----+-----+
| 1 | Alice Johnson | Computer Science | A |
+-----+-----+-----+-----+
1 row in set (0.04 sec)
```

Output:

StudentID	StudentName	Course	Grade
1	Alice Johnson	Computer Science	A

Comparison Table

Aspect	Join Approach	Subquery Approach
How It Works	Combines both tables in one operation using StudentID match	Inner query finds students with >1 enrollment, outer query shows their details
Execution	Single operation (faster)	Two operations (slower)
Result	6 rows - All students with enrollments	1 row - Only students with >1 enrollment
Data Display	Shows data from both tables	Shows data from only Students table
Advantages	Faster, readable, shows all data, good indexing	No duplicates, good for filtering, handles aggregates
Disadvantages	Shows duplicates, can't filter by aggregates directly	Slower, complex, only one table data, temporary storage
Performance	Faster, less CPU/Memory	Slower, more CPU/Memory
Index Usage	Excellent	Limited
Large Datasets	Scales well	Poor performance
Best For	Retrieving related data from multiple tables	Filtering based on aggregates/existence checks

Conclusion

The JOIN approach is faster, more efficient, and easier to read when retrieving related data from multiple tables, making it the preferred choice for displaying information from both tables together. The Subquery approach, while slower and more complex, is better suited for filtering results based on aggregate conditions like COUNT, SUM, or AVG, and for checking existence using EXISTS or NOT EXISTS.

6. Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction.

Bloom's Level: BL6 – Create

View 1: Student Basic Information (Security)**Problem:**

Teachers should see student names and courses but NOT grades.

Query:

```
mysql> CREATE VIEW StudentBasicInfo AS
```

```
-> SELECT
```

```
->   StudentID,
```

```
->   StudentName,
```

```
->   Course
```

```
-> FROM Students;
```

```
mysql> SELECT * FROM StudentBasicInfo;
```

```
mysql> CREATE VIEW StudentBasicInfo AS
-> SELECT
->   StudentID,
->   StudentName,
->   Course
-> FROM Students;
Query OK, 0 rows affected (0.06 sec)

mysql> SELECT * FROM StudentBasicInfo;
+-----+-----+-----+
| StudentID | StudentName | Course |
+-----+-----+-----+
| 1 | Alice Johnson | Data Science |
| 2 | Bob Smith | Mathematics |
| 3 | Charlie Brown | Computer Science |
| 4 | Diana Williams | Physics |
| 5 | Evan Davis | Mathematics |
| 6 | Fiona Miller | Computer Science |
+-----+-----+-----+
6 rows in set (0.00 sec)
```

Output:

StudentId	StudentName	Course
1	Alice Johnson	Data Science
2	Bob Smith	Mathematics
3	Charlie Brown	Computer Science
4	Diana Williams	Physics
5	Evan Davis	Mathematics
6	Fiona Miller	Computer Science

View 2: Enrollment Details (Abstraction)

Problem:

Administrators need to see student enrollment details without writing complex JOIN queries.

Query:

```
mysql> CREATE VIEW EnrollmentDetails AS
-> SELECT
->   s.StudentName,
->   s.Course,
->   en.Subject,
->   en.Credits,
->   en.Semester
-> FROM Students s
-> JOIN Enrollments en ON s.StudentID = en.StudentID;

mysql> SELECT * FROM EnrollmentDetails;
```

```
mysql> CREATE VIEW EnrollmentDetails AS
-> SELECT
->   s.StudentName,
->   s.Course,
->   en.Subject,
->   en.Credits,
->   en.Semester
-> FROM Students s
-> JOIN Enrollments en ON s.StudentID = en.StudentID;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT * FROM EnrollmentDetails;
+-----+-----+-----+-----+-----+
| StudentName | Course | Subject | Credits | Semester |
+-----+-----+-----+-----+-----+
| Alice Johnson | Data Science | Database Systems | 3 | Fall 2025 |
| Alice Johnson | Data Science | Data Structures | 3 | Fall 2025 |
| Bob Smith | Mathematics | Calculus | 4 | Fall 2025 |
| Charlie Brown | Computer Science | Programming | 3 | Fall 2025 |
| Diana Williams | Physics | Mechanics | 4 | Fall 2025 |
| Evan Davis | Mathematics | Statistics | 3 | Fall 2025 |
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Output:

StudentName	Course	Subject	Credits	Semester
Alice Johnson	Data Science	Database Systems	3	Fall 2025
Alice Johnson	Data Science	Data Structures	3	Fall 2025
Bob Smith	Mathematics	Calculus	4	Fall 2025
Charlie Brown	Computer Science	Programming	3	Fall 2025
Diana Williams	Physics	Mechanics	4	Fall 2025
Evan Davis	Mathematics	Statistics	3	Fall 2025

Justification:

For Security:

1. Views hide sensitive data like grades and personal information from unauthorized users
2. Views restrict access to only necessary columns based on user role
3. Views prevent data misuse by limiting what users can see

For Abstraction:

1. Views simplify complex queries by pre-defining Joins and conditions
2. Views hide database complexity from end users
3. Views provide consistent interface even when underlying tables change

Conclusion:

Views are essential for database security and abstraction. They allow restricted data access by hiding sensitive columns like grades from unauthorized users. Additionally, views simplify complex queries by pre-defining Joins and aggregations, making it easier for users to retrieve information without understanding the underlying database structure. For these reasons, views are a powerful tool for implementing security and abstraction in database systems.

7. Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze

Notation:

```
 $\pi$  StudentName (  
   $\sigma$  Course='Computer Science' (Students)  
  ⋈  
   $\sigma$  Subject='Database Systems' (Enrollments)  
)
```

Query:

```
mysql> SELECT DISTINCT s.StudentName  
-> FROM Students s  
-> INNER JOIN Enrollments e ON s.StudentID = e.StudentID  
-> WHERE s.Course = 'Data Science'
```

-> AND e.Subject = 'Database Systems';

```
mysql> SELECT DISTINCT s.StudentName
-> FROM Students s
-> INNER JOIN Enrollments e ON s.StudentID = e.StudentID
-> WHERE s.Course = 'Data Science'
-> AND e.Subject = 'Database Systems';
+-----+
| StudentName |
+-----+
| Alice Johnson |
+-----+
1 row in set (0.00 sec)
```

Output:

StudentName
Alice Johnson

Analyzing the Differences in Representation:

1. Syntax Differences

- Relational Algebra: Uses Greek symbols (π , σ , $\bowtie$) with mathematical notation
- SQL: Uses English keywords (SELECT, WHERE, JOIN) with declarative syntax

2. Join Specification

- Relational Algebra: Natural join ($\bowtie$) implicitly joins on all common attribute names (StudentID in this case)
- SQL: Explicit JOIN condition must be specified (ON s.StudentID = e.StudentID)

3. Order of Operations

- Relational Algebra: Shows explicit operation sequence with nested expressions
- SQL: Declarative structure where conditions can be combined in WHERE clause

4. Projection vs. Selection

- Relational Algebra: π (projection) and σ (selection) are separate, ordered operations
- SQL: SELECT and WHERE combine both operations in a single statement

5. Duplicate Handling

- Relational Algebra: Projection (π) by default doesn't eliminate duplicates unless specified
- SQL: SELECT returns duplicates unless DISTINCT is specified

Conclusion:

The relational algebra expression and SQL query both achieve the same result, with SQL providing a more user-friendly, declarative approach compared to the procedural mathematical notation of relational algebra.

8. Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate

Inefficient Query:

```
mysql> -- INEFFICIENT QUERY: Using subquery with IN and multiple scans
mysql> SELECT s.StudentName, s.Course, s.Grade
-> FROM Students s
-> WHERE s.StudentID IN (
->     SELECT e.StudentID
->     FROM Enrollments e
->     WHERE e.Subject = 'Database Systems'
-> )
-> AND s.Course IN (
->     SELECT DISTINCT Course
->     FROM Students
->     WHERE Grade = 'A'
-> );
```

StudentName	Course	Grade
Alice Johnson	Data Science	A

1 row in set (0.00 sec)

Performance Issues:

- Multiple subquery executions
- No proper indexing
- Scans entire Students table multiple times
- IN clause with subquery causes nested loop operations

Using Join:

```
mysql> -- OPTIMIZED QUERY 1: Using INNER JOIN
mysql> SELECT s.StudentName, s.Course, s.Grade
-> FROM Students s
-> INNER JOIN Enrollments e ON s.StudentID = e.StudentID
-> WHERE e.Subject = 'Database Systems'
-> AND s.Grade = 'A';
```

StudentName	Course	Grade
Alice Johnson	Data Science	A

1 row in set (0.00 sec)

Using View:

```
mysql> -- Create VIEW for reusable query
mysql> CREATE VIEW GradeA_DatabaseStudents AS
  -> SELECT s.StudentID, s.StudentName, s.Course, s.Grade, e.Subject
  -> FROM Students s
  -> INNER JOIN Enrollments e ON s.StudentID = e.StudentID
  -> WHERE e.Subject = 'Database Systems'
  -> AND s.Grade = 'A';
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> -- Simplified query using VIEW
mysql> SELECT StudentName, Course, Grade
  -> FROM GradeA_DatabaseStudents;
+-----+-----+-----+
| StudentName | Course      | Grade |
+-----+-----+-----+
| Alice Johnson | Data Science | A      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Using Index:

```
mysql> -- Create indexes for faster searches
mysql> CREATE INDEX idx_students_grade ON Students(Grade);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_enrollments_subject ON Enrollments(Subject);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_enrollments_studentid ON Enrollments(StudentID);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql>
mysql> -- Query now uses indexes automatically
mysql> SELECT s.StudentName, s.Course, s.Grade
  -> FROM Students s
  -> INNER JOIN Enrollments e ON s.StudentID = e.StudentID
  -> WHERE e.Subject = 'Database Systems'
  -> AND s.Grade = 'A';
+-----+-----+-----+
| StudentName | Course      | Grade |
+-----+-----+-----+
| Alice Johnson | Data Science | A      |
+-----+-----+-----+
1 row in set (0.00 sec)
```

Performance Comparison Table

Criteria	Original Query (Subquery)	Using JOIN	Using VIEW	Using INDEXES
Table Scans	3	1	1	1
Index Usage	No	No	No	Yes
Disk I/O	High	Medium	Medium	Low
Query Complexity	Complex	Simple	Very Simple	Simple
Reusability	Low	Medium	High	Medium
Maintainability	Poor	Good	Excellent	Good
Scalability	Poor	Good	Good	Excellent

Conclusion:

JOINS reduce query complexity and improve execution time by eliminating inefficient subqueries. INDEXES provide the most significant performance boost by enabling fast index seeks instead of full table scans, while VIEWS enhance code reusability and maintainability.

9. Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create

Problem:

Find students who have taken at least 2 courses and have an 'A' grade, showing their total courses and ranking them by performance

Query & Output:

```
mysql> SELECT
->     s.StudentName,
->     s.Course,
->     s.Grade,
->     (SELECT COUNT(*)
->      FROM Enrollments e
->      WHERE e.StudentID = s.StudentID) AS TotalCourses
-> FROM Students s
-> WHERE s.Grade = 'A'
->    AND (SELECT COUNT(*)
->         FROM Enrollments e
->         WHERE e.StudentID = s.StudentID) >= 2
-> ORDER BY s.Grade DESC;
```

StudentName	Course	Grade	TotalCourses
Alice Johnson	Data Science	A	2

1 row in set (0.00 sec)

Conclusion

The query successfully identifies top-performing students by filtering for 'A' grade students with at least 2 course enrollments, demonstrating the use of correlated subqueries to dynamically count enrollments per student. Alice Johnson from Data Science is the only student meeting both criteria, showing how nested queries can effectively filter and aggregate data for academic performance analysis.

10. Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate

Scenario:

Management needs daily reports of student performance showing student names, their course, grade, and total number of courses enrolled.

Using JOIN

```
mysql> SELECT
->     s.StudentName,
->     s.Course,
->     s.Grade,
->     COUNT(e.EnrollmentID) AS TotalCourses
-> FROM Students s
-> LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
-> GROUP BY s.StudentID, s.StudentName, s.Course, s.Grade;
```

StudentName	Course	Grade	TotalCourses
Alice Johnson	Data Science	A	2
Bob Smith	Mathematics	B	1
Charlie Brown	Computer Science	C	1
Diana Williams	Physics	A	1
Evan Davis	Mathematics	B	1
Fiona Miller	Computer Science	A	0

6 rows in set (0.00 sec)

Using VIEW

```
mysql> -- Create VIEW once
mysql> CREATE VIEW StudentPerformance AS
-> SELECT
->     s.StudentName,
->     s.Course,
->     s.Grade,
->     COUNT(e.EnrollmentID) AS TotalCourses
-> FROM Students s
-> LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
-> GROUP BY s.StudentID, s.StudentName, s.Course, s.Grade;
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> -- Simple query using VIEW
mysql> SELECT * FROM StudentPerformance;
```

StudentName	Course	Grade	TotalCourses
Alice Johnson	Data Science	A	2
Bob Smith	Mathematics	B	1
Charlie Brown	Computer Science	C	1
Diana Williams	Physics	A	1
Evan Davis	Mathematics	B	1
Fiona Miller	Computer Science	A	0

6 rows in set (0.00 sec)

Using Subquery:

```
mysql> SELECT
->     s.StudentName,
->     s.Course,
->     s.Grade,
->     (SELECT COUNT(*)
->      FROM Enrollments e
->      WHERE e.StudentID = s.StudentID) AS TotalCourses
-> FROM Students s;
```

StudentName	Course	Grade	TotalCourses
Alice Johnson	Data Science	A	2
Bob Smith	Mathematics	B	1
Charlie Brown	Computer Science	C	1
Diana Williams	Physics	A	1
Evan Davis	Mathematics	B	1
Fiona Miller	Computer Science	A	0

6 rows in set (0.00 sec)

Comparison Table

Criteria	JOIN	VIEW	Subquery
Code Reusability	Low (rewrite daily)	High	Low (rewrite daily)
Maintainability	Moderate	Excellent	Poor
Query Complexity	Simple	Very Simple	Complex
Performance	Good	Best	Poor
Security	Basic	High	Basic
Abstraction	None	Excellent	None

Suitable For	One-time use	Daily reports	Existence checks
--------------	--------------	---------------	------------------

Conclusion:

For daily management reports requiring frequent access, VIEW is the optimal choice due to its reusability, simplified access, and maintainability, while JOIN would require rewriting the query daily and Subquery would be inefficient with correlated executions.

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate;	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly Interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or Unclear interpretation	No interpretation